Parsing a musical piece ?

I The Idea

We want to find a likely chord progression from a given piece.

II Some precisions

II.A Definitions

II.A.a Grammar

We will use a context-free grammar, where non terminal elements are denoted by majuscules Latin letters and terminals by minuscules Latin letters.
Non-terminal and terminal elements represent chords, and are kept as a pair (root pitch, chord quality).
Note that we associate a certain weight to each rule.

If we have an abstract free grammar :
We suppose that for all X, Y, Z, there is at most one rule doing X -> Y Z . If not, we only keep one of the most cheaper ones (makes a context-free grammar back)

II.A.b Notations

Let n be the number of bar of the piece.
Let f be a 'fitting function'.
Let s be a sequence of n terminal elements.
Let w : *rule* $\rightarrow$ *associated weight* be the weight function on the context-free grammar.
Let $\mathbb{P}_i$: a $\rightarrow \mathbb{P}_i$(a) the function associating each terminal element to its probability to be on the i-th measure, for all $1 \leq i \leq n$ (See II.B.a).
Let T be a tree generating the sequence s in the context-free grammar.
A sequence s as above is said to be feasible if $\mathbb{P}_i(s[i]) \neq 0, \forall 1 \leq i \leq n$.

II.B The interesting (sub)problem

II.B.a What we have

Suppose for now there is only one chord per bar in the chord progression we want to generate.
Suppose we already extracted from the piece, for each measure, a set of chords (as terminal elements) that could potentially fit in it, with some probabilities (described by the $\mathbb{P}_i$'s).
We thus have n spaces, and for each space potential terminal elements with associated probabilities, as well as our previously mentionned context-free grammar

II.B.b What we want

We want to be able to reconstruct the best-fitting tree T that spans a sequence s of n terminal elements such that each i-th element of this sequence is in the set of potential terminal elements (with non-zero probability) of the i-th measure.
The probabilities of the terminal elements and the weight of the tree must be taken into account to determine the fitting of the tree.

The probabilities of terminal elements can be seen as how well the tree fit into the piece and the weight of the tree as how well the tree fit into music in general.

II.B.c What we need

1. A fitting function f for a tree that uses, as stated above, the probabilities of the terminal elements and the weigth of the tree.
2. An (hopefully polynomial) algorithm to find or construct the best-fitting tree based on this fitting function f.

III Reflections

III.A Definitions that may be useful

III.A.a Probability of a sequence

Let s as above. The probability associated to s is the product of probabilities of elements of s, denoted by $\mathbb{P}(s)$. The unlikelihood of s is $\Pi_{i=1}^{n}(1-\mathbb{P}_i(s[i]))$, denoted by $U(s)$.

III.B Remarks

1. When a sequence s contain a terminal element with associated probability 0, all its spanning tree T should be considered not fitting
2. The tree T should fit better as the probability of s increases and as its weight decreases
3. The weight of a tree T is not defined yet, but can be seen as the sum of all its rules weights for now.
4. Thus, construct the best-fitting T for each possible s is an angle of approach

IV We can thus rephrase the problem as such :

We suppose that from the piece have already been extracted the possible chords from each measure, with probabilities.

IV.A Notations

Let n be the number of bars of the piece.

IV.B Inputs

1. A context-free grammar Γ in Chomsky normal form, where non-terminal and terminal elements represent chords, and are kept as a pair (root pitch, chord quality). Let T be its set of terminal elements, and $m = \#\mathrm{T}$. Let U be its set of non-terminal elements. Let Δ be its set of rules. Let S be its starting element.
2. A weight function $w : \Delta \to \mathbb{R}^+$
3. A matrix $A \in \mathbb{R}^{n\times\,\mathrm{T}}$, A_{ij} representing the probability associated to chord j being present in measure i.

IV.C Definitions

IV.C.a Grammar Trees

A grammar tree of such a grammar Γ, for us, will be a rooted ordered tree satisfying :

1. All vertices v have an associated value $X_v \in \mathrm{T} \cup \mathrm{U}$
2. The root r verifies $X_r = S$
3. For each leaf v, $X_v \in \mathrm{T}$
4. For each non-leaf vertex v, $X_v \in \mathrm{U}$
5. If the children of a node u are $v^1, ..., v^k$, in this order, then $\left(X_u \to X_{v^1} \ ... \ \ X_v^k\right) \in \Delta$

Let $\mathbb{T}$ denote the set of grammar trees.
Let $\mathbb{T}^*$ be the set of trees satisfying all conditions except 2. ($X_r \neq S$ is accepted).

We say that $T \in \mathbb{T}$ is a spanning tree of $s \in \mathrm{T}^n$ if taking T leave's values in the natural order given by the ordered tree gives us s.

IV.D Needs

1. A certain function $f : \mathbb{T} \to \mathbb{R}^+$ which sends a tree T to some measure of its fittingness to the piece
2. The function $g : \mathrm{T}^n \to \mathbb{R}^+, \quad s \mapsto \min_{T \text{ spanning tree of } s} f(T)$ which sends a string of terminal elements to the lowest value of f over its spanning trees

Note that $g(s)$ should be "infinite" if s is not in the language generated by Γ.
Moreover, if one column of A is null, $g(s)$ should be "infinite" for all s.

IV.E Outputs

A string $s \in \mathrm{T}^n$ of terminal elements such that $g(s)$ is minimal

The Problem is then equivalent to find

$\min_{s \in \mathrm{T}^n} g(s)$,

where f is yet to define.

V Subproblem

V.A Goal

The goal is to find if, from a sequence l of n sets of terminal elements, there exists a string s of n terminal elements such that

1. $s[i] \in l[i], \forall 1 \leq i \leq n$
2. s is in the language generated by the grammar Γ

Such a string s is called a *fitting* string for l.
The reduction is that basically, we do not take the weights into account anymore and just want to find if a feasible string exists.

V.B Input

A grammar Γ as described in IV.B.
A sequence l of set[T], of length n.
It is also possible to get the same input and just take the informations $\mathbb{1}_{\{A_{ij} \neq 0\}}$ from A.
The sequence l described above is then given by $l[i] = \{\mathrm{X} \in \mathrm{T} \mid A_{iX} \neq 0\}$.

V.C Output

The boolean $\mathbb{1}_{\exists s}$ as described in V.A

V.D Algorithm

Since the grammar Γ is in Chomsky normal form, we can actually apply the CYK algorithm with a different starting step.
We use the notation $P[i,j]$ for the set of non-terminal elements that can generate a sequence of length $j-i+1$ that fits $l[i:j]$.
Instead of defining $P[i,i]$ as $\{\mathrm{N} \in \mathrm{U} \mid (\mathrm{N} \to \text{the } i-\text{th terminal element of the word}) \in \Delta\}$, since we have a set of possible terminal elements at each step i, we just define

$$P[i,i] = \{\mathrm{N} \in \mathrm{U} \mid \exists\ \mathrm{a} \in l[i] \subseteq \mathrm{T}\ s.t.\ (\mathrm{N} \to \mathrm{a}) \in \Delta\}$$

Then the rests of the $P[i,j]$ are computed by induction on $|j-i|$, with the same recurrence relation as in CYK:
for all $1 \leq i < j \leq n$,

$$P[i,j] =$$
$$\{\mathrm{N} \in \mathrm{U} \mid \exists\ \mathrm{B}, \mathrm{C} \in \mathrm{U},\ i \leq k \leq j-1,\ \text{such that } \mathrm{B} \in P[i,k],\ \mathrm{C} \in P[k+1,j],\ \text{and } (\mathrm{N} \to \mathrm{B}\ \mathrm{C}) \in \Delta\}$$

We return $\mathbb{1}_{S \in P[1,n]}$.

V.E What this opens

With the (very very) high similarity of our algorithm with CYK, we see that the general problem will probably be as difficult to solve as having to put weights on the CYK algorithm.

VI Putting weights on CYK

VI.A The Concept

Let $s \in \mathrm{T}^n$.
Conceptually, we want to use CYK to get not only the existence of a tree, nor the different possible trees, spanning s, but the minimal weight tree spanning s, or at least its weight.

VI.B The Idea

Suppose that from any tree $\mathrm{T} \in \mathbb{T}^*$, we can get its weight in $O(1)$ from the weights of the subtrees whose roots are the root's children, and the grammar rule corresponding to
(the roots value $\to$ its children's values, in order), with say the function W.
Also suppose W is an increasing function of theses weights. This reduction is reasonable, as we do not want heavier subtrees giving a lighter total tree.
Note that usually, W will be a simple operation on the two weights and the weight of the rule (given by the function w), like sums, products or maxima.

We want to get the $Q[i,j],\ 1 \leq i \leq j \leq n$, which will basically be the $P[i,j]$ with a 'minimal weight' information added.

What we do when computing $P[i,j]$, is instead of only getting the $\mathrm{N} \in \mathrm{U}$ which verifies the condition we need, we also get the minimal weight w_{N} of a tree spanning $s[i:j]$ with N as roots.
The wanted information is then (N, w_{N}).

For $P[i,i]$, we get :

$$Q[i,i] = \{(\mathrm{N}, w_{\mathrm{N}}) \in \mathrm{U} \times \mathbb{R}^+ \mid (\mathrm{N} \to s[i]) \in \Delta\},$$

where w_{N} is the weight of the tree with root N and only leaf $s[i]$, and no other vertex.

For $1 \leq i < j \leq n$, we start from

$$P[i,j] = \{\mathrm{N} \in \mathrm{U} \mid \exists\ \mathrm{B}, \mathrm{C} \in \mathrm{U},\ i \leq k \leq j-1, \text{ such that } \mathrm{B} \in P[i,k],\ \mathrm{C} \in P[k+1,j], \text{ and } (\mathrm{N} \to \mathrm{B}\ \mathrm{C}) \in \Delta\}.$$

For each N in that, we want the minimal weight of a tree that indeed generates $s[i:j]$ with N as root value.
We know that minimal weight for all $\mathrm{B} \in P[i,k],\ \mathrm{C} \in P[k+1,j],\ i \leq k \leq j-1$.
If $\alpha \in Q[i,k]$ such that $\mathrm{B} = \alpha[0], \beta \in Q[k+1,j]$ such that $\mathrm{B} = \beta[0]$,
let $w_\alpha = \alpha[1]$ and $w_\beta = \beta[1]$.

Thus, for fixed root's children (of values B and C respectively), and fixed size of the corresponding subtrees ($k-i+1$ and $j-k$ respectively), the minimal weight possible has to be $W\big(w_\alpha, w_\beta\big)$

VI.C The Algorithm

The construction of $Q[i,j]$ goes then as follows :

for $k \in [\![i, j-1]\!]$:
    for $(\alpha, \beta) \in Q[i,k] \times Q[k+1,j]$:
        let $\mathrm{B} = \alpha[0]$ and $\mathrm{C} = \beta[0]$
        if $(\mathrm{N} \to \mathrm{B}\ \mathrm{C}) \in \Delta$:
            if N is already in $Q[i,j][0]$: *### already a pair (N, w_{N}) added to $Q[i,j]$*
                if $W\big((\mathrm{N} \to \mathrm{B}\ \mathrm{C}), w_\alpha, w_\beta\big) < w_{\mathrm{N}}$:
                    $Q[i,j]$.del$(\mathrm{N}, w_{\mathrm{N}})$ *### replace the ancient weight by the better one*
                    $Q[i,j]$.add$\big(\mathrm{N}, W\big((\mathrm{N} \to \mathrm{B}\ \mathrm{C}), w_\alpha, w_\beta\big)\big)$
            else :
                $Q[i,j]$.add$\big(\mathrm{N}, W\big((\mathrm{N} \to \mathrm{B}\ \mathrm{C}), w_\alpha, w_\beta\big)\big)$
    end
end
if S is in $Q[1,n][0]$:
    let t be such that $(S,t) \in Q[1,n]$ *### normally only one such t*
    return t

else :

return None

VI.D The Remarks

The $Q[i, j]$'s can also be structured as dictionaries for a probably better implementation.
Iterating on the rules and not the elements composing them may result in less iterations.

VII Putting weights on the subproblem

VII.A Concept

Conceptually, we want to put weights on the algorithm in **V** like we did on CYK in **VI** to get the minimal weight, then tweak the algorithm a little to also output the associated string.

In fact, we will do the opposite, i.e. tweak the first step of the algorithm in **VI** like in **V** to get what we want.

VII.B Inputs

The input is the same as in the subproblem **V**, plus the weight function W used in **VI**.

VII.C Get the minimal weight

VII.C.a First step's modification

Mixing the ideas in parts **V** and **VI**, we get

$$Q[i, i] = \{(\mathrm{N}, w_{\mathrm{N}}) \in \mathrm{U} \times \mathbb{R}^+ \mid \exists\ \mathrm{a} \in l[i], \text{ s.t. } (\mathrm{N} \to \mathrm{a}) \in \Delta\}$$

for all $1 \leq i \leq n$.

Then, we can get all the others $Q[i, j]$ with the recurrence relation as before, thus getting at the end the smallest weight of a tree that generates a string fitting the sequence l.
We now still need to be able to output this said string.

VII.D Output the string of minimal weight

We will base ourselves on the computation of the $Q[i, j]$'s to get $R[i, j]$'s that additionally keep track of which string do the trees of minimal weights generate.

VII.D.a first step

Here we show how to construct $R[i, i]$ for all $1 \leq i \leq n$.
The idea is to keep track of the terminal elements that N generates, as in

$$\{(\mathrm{N}, w_{\mathrm{N}}, \mathrm{a}) \in \mathrm{U} \times \mathbb{R}^+ \times \mathrm{T} |\ \mathrm{a} \in l[i],\ (\mathrm{N} \to \mathrm{a}) \in \Delta\}$$

But we only want to get at most one value in $R[i,i]$ for each $\mathrm{N} \in \mathrm{U}$ (the minimal weight one).
We can explicit the algorithm as in **VI.C** :

for (N, a) in $\mathrm{T} \times l[i]$ such that $(\mathrm{N} \to \mathrm{a}) \in \Delta$:

if N already is in $R[i,i][0]$: ### *the element* N *is in is* $(\mathrm{N}, w_{\mathrm{N}}, \mathrm{b}) \in R[i,i] \subseteq \mathrm{U} \times \mathbb{R}^+ \times \mathrm{T}$

if $w_{(\mathrm{N},\ \mathrm{a})}$ is strictly less than w_{N} :
$w_{(\mathrm{N},\ \mathrm{a})}$ *is the weight associated to the tree with* N *as root and* a *as only other vertex*

$R[i,i].\mathrm{del}(\mathrm{N}, w_{\mathrm{N}}, \mathrm{b})$ ### *replace the ancient weight by the better one*

$R[i,i].\mathrm{add}\big(\mathrm{N}, w_{(\mathrm{N},\ \mathrm{a})}, \mathrm{a}\big)$

else :

$R[i,i].\mathrm{add}\big(\mathrm{N}, w_{(\mathrm{N},\ \mathrm{a})}, \mathrm{a}\big)$

VII.D.b Induction step

Here we show how to construct $R[i,j]$ for all $1 \leq i < j \leq n$, by induction on $|j-i|$.

Idea :
Suppose that we are adding $\big(\mathrm{N}, W\big((\mathrm{N} \to \mathrm{B}\ \mathrm{C}), w_\alpha, w_\beta\big)\big)$ in $Q[i,j]$ like in **VI.C**, with $(\alpha, \beta) \in Q[i,k] \times Q[k+1,j]$
for some $k \in [\![i, j-1]\!]$.
Using $R[i,k]$ and $R[k+1,j]$, we also have the information of what strings s and t fitting for $l[i:k]$ and $l[k+1:j]$ respectively, and with $\alpha[0]$ and $\beta[0]$ as roots values respectively, are generated with a minimal cost .
Then we know that the string fitting for $l[i:j]$, with N as root value, and with $\alpha[0]$ and $\beta[0]$ as values of the root's children (in this order), that is generated with a minimal cost,
is $s+t$ (concatenation of strings).

Algorithm : We then can copy the algorithm from **VI.C** with some minor changes described above.

for $k \in [\![i, j-1]\!]$:

for $(\alpha, \beta) \in R[i,k] \times R[k+1,j]$:

let $\mathrm{B} = \alpha[0]$ and $\mathrm{C} = \beta[0]$

if $(\mathrm{N} \to \mathrm{B}\ \mathrm{C}) \in \Delta$:

if N is already in $R[i,j][0]$: ### *already a tuple (*$\mathrm{N}, w_{\mathrm{N}}, s_{\mathrm{N}}$*) added to* $Q[i,j]$

if $W\big((\mathrm{N} \to \mathrm{B}\ \mathrm{C}), w_\alpha, w_\beta\big) < w_{\mathrm{N}}$:

$R[i,j].\mathrm{del}(\mathrm{N}, w_{\mathrm{N}}, s_{\mathrm{N}})$ ### *replace the ancient weight by the better one*

$R[i,j].\mathrm{add}\big(\mathrm{N}, W\big((\mathrm{N} \to \mathrm{B}\ \mathrm{C}), w_\alpha, w_\beta\big), \alpha[2] + \beta[2]\big)$

else :

$R[i,j].\mathrm{add}\big(\mathrm{N}, W\big((\mathrm{N} \to \mathrm{B}\ \mathrm{C}), w_\alpha, w_\beta\big), \alpha[2] + \beta[2]\big)$

end

end

if S is in $R[1,n][0]$:

let (p,t) be such that $(S,p,t) \in R[1,n]$ *### normally only one such pair* (p,t)

return (t,p)

else :

return None

VII.D.c Remark

Note that

$$R[i,j] \subset U \times \mathbb{R}^+ \times \mathrm{T}^{|j-i+1|}, \forall 1 \leq i \leq j \leq n$$